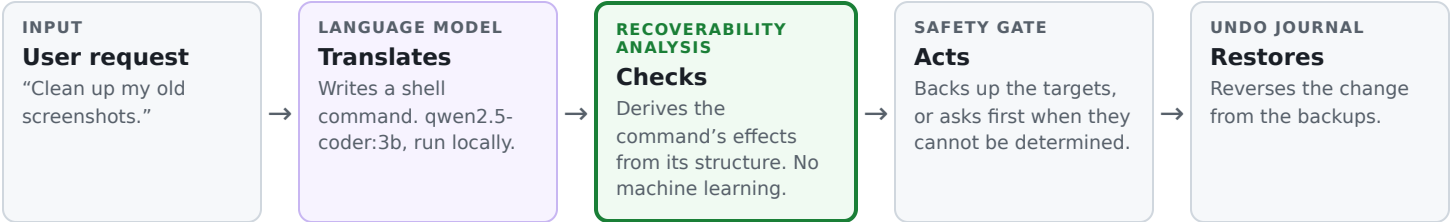


How the Recoverability Analysis Works

SynapseOS has two separate components. A local language model translates a request into a shell command; the recoverability analysis then checks that command before it runs: what it will change, and whether it can be undone. **Only the translation uses AI. The analysis itself does not use the language model:** it is deterministic program analysis that checks the model's output, specified in Chapter 3, §2.1(c) (pp. 62–66).

Where the analysis sits



■ **Probabilistic:** the same request may yield different commands. ■ **Deterministic:** the same command always yields the same decision.

Inside the analysis, in five steps

Running example: `find . -name "Screenshot*.png" -delete`

1	Parse Reads the command's structure with a shell-language parser.	Program <code>find</code> ; search path <code>.</code> ; filter <code>-name "Screenshot*.png"</code> ; action <code>-delete</code> .
2	Look up A rule table records what each program does to files.	<code>find ... -delete</code> removes every file that matches its filter; which files those are is not named in the command.
3	Compose Sees through searches, pipes, loops and wrappers to the program that acts.	The deletion is carried out by <code>find</code> itself on files it discovers while running. The same reasoning applies to <code>find -exec rm</code> , <code> xargs rm</code> , <code>for</code> loops and <code>sh -c</code> .
4	Resolve Runs only the proven read-only part, in a sandbox where nothing can be written.	ILLUSTRATION (SIMPLIFIED) OF AN INTERNAL STEP; USERS NEVER SEE IT: <pre>\$ find . -name "Screenshot*.png" # -delete replaced by a read-only print ./Screenshot 2026-09-20 184502.png ./Screenshot 2026-09-14 101211.png</pre>
5	Decide Classifies the effects and plans what to back up.	ACTUAL ANALYSIS OUTPUT, UNEDITED: <pre>command: find . -name "Screenshot*.png" -delete composition: enabled (full algorithm) class: recoverable-with-capture remove ./Screenshot 2026-09-20 184502.png remove ./Screenshot 2026-09-14 101211.png plan: what is captured before it runs trash ./Screenshot 2026-09-14 101211.png trash ./Screenshot 2026-09-20 184502.png</pre> Both files are backed up before the command runs: <i>trash</i> keeps a hard link to each, so the backup takes no extra space. Had their effects been undeterminable (an unknown program, inline Python, <code>shred</code>), the command would be classified <i>unrecoverable</i> and confirmation required.

Why it matters

It protects data, not intent The decision is made by code, not a model, so a wrong command that would destroy files is still caught. It cannot tell whether a command is what you meant: a wrong but harmless one runs. That part is only as good as the model.	What it improves on Coding agents check commands against a list of dangerous patterns and rely on git to undo mistakes. Personal files have no git, and a list misses targets hidden in <code>find</code> or <code>xargs</code> . On 777 test commands (§3.1(g)), a list let 41% of destructive ones run unprotected; this analysis let none, and backed up 89.5%.	It is the computer-science contribution The analysis is static program analysis in the tradition of abstract interpretation (pp. 25–26), not machine learning. The model is the interface; the analysis is the research.
---	--	--

Inside the Algorithm

The algorithm in pseudocode, condensed from Algorithm 1 in Chapter 3, §2.1(c) (p. 66), and where each part lives in the code. Everything here is deterministic program analysis; none of it calls the language model.

Pseudocode

```

Input: command c, working directory wd, capture mechanisms M
Output: verdict  $\in \{\text{RECOVERABLE}, \text{RECOVERABLE\_WITH\_CAPTURE}, \text{UNRECOVERABLE}\}$ , and plan P
State: overlay V of paths created or removed by earlier parts of c

function EFFECTS(node, wd, V)                                // what the command does to files
  if node is a single command then return RULE(name, args, wd, V) // unknown command → {UNKNOWN}
  if node = A ; B, A && B, A || B, or A | B then
    return EFFECTS(A) ∪ EFFECTS(B)                          // B sees what A did, through V
  if node writes to a redirect target t then return EFFECTS(rest) ∪ {write-or-create(t)}
  if node = for x in W; do body then return  $\cup_{w \in \text{EXPAND}(W)} \text{EFFECTS}(\text{body}[x:=w])$ 
  if node is sh -c s, or a wrapper (sudo, env, ...) then return EFFECTS(inner command)
  if node is find -delete / -exec, or xargs then
    T ← RESOLVE(node); if T = ⊥ then return {UNRESOLVED}
    return  $\cup_{t \in T} \text{EFFECTS}(\text{action}[\{\} := t])$ 

function RESOLVE(node)                                        // learns run-time targets, changing nothing
  d ← DRYRUN(node)                                           // find with actions replaced by prints; xargs' producer; tar -t, git clean -n ...
  if EFFECTS(d) ≠ ∅ or d cannot run then return ⊥           // only a proven read-only d is ever run
  return the targets d prints

function VERDICT(E)                                          // fail closed: unknown always asks
  if UNRESOLVED ∈ E or UNKNOWN ∈ E or some effect cannot be captured then return UNRECOVERABLE
  if every effect is a create or a move then return RECOVERABLE
  return RECOVERABLE_WITH_CAPTURE

function PLAN(E, M)                                         // the cheapest backup that suffices
  P ← ∅
  for each path p removed, overwritten, or re-permissioned in E, in path order do
    if p lies under a directory already in P then continue
    P ← P ∪ {(cheapest m ∈ M that restores p, p)} // hard link | copy | permission record
  return P

```

File map: `prototype/internal/`

```

prototype/
└─ internal/
   └─ effects/           the algorithm
      └─ types.go        effects, issues, the result
      └─ analyze.go      walk, overlay V (EFFECTS)
      └─ wrappers.go     sudo, sh -c, find, xargs
      └─ rules.go        per-command rules (RULE)
      └─ adapters.go     dry runs (RESOLVE)
      └─ runner.go       read-only sandbox
      └─ pkgstate.go     packages, services
      └─ verdict.go      VERDICT, PLAN
      └─ failclosed.go   baseline: fail-closed list
      └─ catalog.go      rule notes for the docs
      └─ *_test.go       6 files, 53 tests, fuzz
   └─ gate/             uses it ask, or back up
   └─ undo/             uses it journal, restore
   └─ classifier/       evaluates it pattern list
   └─ baselines/        evaluates it Codex CLI
   └─ oracle/           evaluates it ground truth
   └─ recoverycheck/    evaluates it undo check
└─ ollama/ toolcall/ executor/ session/
   └─ loopevent/ telemetry/ tui/ typedops/
      └─ rest of SynapseOS

```

Guarantees, and how they are checked

- **Fails closed.** Anything it cannot determine is *UNRESOLVED* or *UNKNOWN*, which can only push the verdict to asking.
- **Changes nothing while analysing.** A fuzz test fed 336,639 random command lines; every command the analysis ran was read-only.
- **Deterministic.** The same command always gets the same answer; no AI in this code.
- **Measured.** 53 test functions, 85% of the analysis covered, enforced by `make ci`; evaluated on 777 commands (Chapter 3, §3.1).

A request through the code: model writes *c* → `gate.Decide` → `effects.Analyze` → **VERDICT, PLAN** → ask, or `gate.Capture` → run → `synapse undo`

Full reference: `docs/recoverability-analysis.md`